

# CoachApp — Developer Handover

---

**Audience:** a developer taking over the backend. This explains the stack, the architecture, every entity, the auth/tenancy model, the coach-side and trainee-side services, the cross-cutting rules, and how to build/run/extend the project.

**Golden rule of this codebase:** \*the code is the spec.\* When any doc disagrees with the code, the code wins. The reference slice to imitate is **Trainee / WorkoutPlan** under `src/`.

---

## 1. Stack & shape

---

- **Framework:** ABP Framework **10.4.0** on **.NET 10**.
- **Template:** ABP single-application (`app`) template — **not** the modular template. There is

no `modules/` folder; everything lives under `src/`.

- **Style:** pragmatic, **feature-folder** vertical slices (modeled on `../JasimExpress/express_api`).
- **Database:** SQL Server, schema managed by **EF Core migrations** (never manual SQL).
- **Auth:** OpenIddict (auth server + token validation in the same host process).
- **Multi-tenancy:** enabled — **one coach = one tenant**.
- **Mapping:** **Mapster** (no AutoMapper/Mapperly).
- **API:** ABP **conventional (auto) controllers** — AppServices are exposed as REST automatically;

there are no hand-written controllers.

- **Not used (deliberately):** MediatR/CQRS, AutoMapper, Mapperly, FluentValidation, custom

repositories for plain CRUD, `*Factory/*Manager` classes for simple CRUD.

⚠ **Naming quirk:** the folder and namespace segment is spelled **Entites** (missing the second **i**). It is intentional and consistent everywhere — entity + DTOs + interface live under `CoachApp.Entites.<Feature>`. Don't "fix" it; you'll break the build.

---

## 2. Solution layout

| Project                                       | Responsibility                                                                                                                                                                                      |
|-----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>CoachApp.Domain.Shared</code>           | Consts ( <code>Entites/&lt;Feature&gt;/</code> ), enums ( <code>Enums/</code> ), <code>CoachAppDomainErrorCodes</code> , roles/prefixes, localization ( <code>Localization/CoachApp/*.json</code> ) |
| <code>CoachApp.Domain</code>                  | Entities ( <code>Entites/&lt;Feature&gt;/</code> ), data seeders, OpenIddict seeder, the single-coach guard                                                                                         |
| <code>CoachApp.Application.Contracts</code>   | DTOs + <code>I&lt;Entity&gt;AppService</code> ( <code>Entites/&lt;Feature&gt;/</code> ), permissions ( <code>Permissions/</code> )                                                                  |
| <code>CoachApp.Application</code>             | AppServices ( <code>Apis/&lt;Feature&gt;/</code> ), <code>CoachAppMapsterConfig</code> , <code>MapsterObjectMapper</code> , <code>MyTraineeAppServiceBase</code>                                    |
| <code>CoachApp.EntityFrameworkCore</code>     | <code>CoachAppDbContext</code> , <code>EntityConfigurations/&lt;Feature&gt;/</code> , <code>Migrations/</code>                                                                                      |
| <code>CoachApp.HttpApi / .HttpApi.Host</code> | Auto controllers, host wiring, Swagger, auth                                                                                                                                                        |
| <code>CoachApp.DbMigrator</code>              | Console app: applies EF migrations + seeds roles/permissions/admin                                                                                                                                  |

### Namespace convention (the layer name is dropped)

- Entity + DTOs + `I<Entity>AppService` → `CoachApp.Entites.<Feature>`
- AppService → `CoachApp.Apis.<Feature>`
- Enums → `CoachApp.Enums`
- EF config → `CoachApp.EntityConfigurations.<Feature>`
- Permissions → `CoachApp.Permissions`
- Error codes → `CoachApp` (`CoachAppDomainErrorCodes`)

`<Feature>` is the pluralized entity name (`Trainees`, `WorkoutPlans`, ...).

---

### 3. How a request flows

---

HTTP request

- ABP conventional controller (auto-generated from the AppService interface)
- [Authorize] permission check (class-level + per-method)
- AppService (Apis/<Feature>) – the "fat" layer:
  - resolve current user/tenant (ICurrentUser / ICurrentTenant)
  - query via IRepository<T,Guid>.GetQueryableAsync() + WhereIf + Dynamic.Core
- OrderBy + Skip/Take + AsyncExecuter
  - cross-aggregate existence/uniqueness checks → throw BusinessException(errorCode)
  - mutate via the aggregate's static Create(...) / behavior methods (Add\*, Clear\*, Activate...)
  - persist via IRepository<T,Guid>
  - map entity → DTO with ObjectMapper.Map (routed through Mapster)
  - enrich DTOs with cross-aggregate display data (exercise/food names, macro totals)
- DTO response (auto-serialized)

Multi-tenancy is applied in the pipeline (UseMultiTenancy, after authentication), so every IMultiTenant query is automatically filtered to the current tenant — you rarely filter by TenantId by hand.

---

### 4. Domain model

---

#### 4.1 Conventions across all entities

- **Aggregate roots:** FullAuditedAggregateRoot<Guid> + IMultiTenant (public virtual Guid?

TenantId). Gives Id, creation/modification audit, **soft delete**, and tenant isolation.

- **Child (owned) entities:** Entity<Guid>. Their back-reference FK to the parent is protected set (set only via constructor). They are created **only** through the root's Add\* methods — that's the aggregate boundary.

- **Validation:** via ABP Check.NotNullOrWhiteSpace / Check.Length against the shared \*Consts

(throws ArgumentException). The entities themselves throw **no** BusinessException — real cross-aggregate invariants (single active plan, "max 100 entries", references exist) are enforced in the **application layer**.

- **Factory + behavior:** each root has a `static Create(Guid id, ..., Guid? tenantId = null);` state

changes go through behavior methods (`Add*`, `Clear*`, `Activate/Deactivate`). No `*Factory/*Manager`.

## 4.2 Aggregate map

|                           |                                                                 |                      |
|---------------------------|-----------------------------------------------------------------|----------------------|
| Trainee                   | (root; links to IdentityUser via UserId)                        | — standalone         |
| Exercise                  | (root; coach's exercise library)                                | — standalone         |
| Food                      | (root; coach's food library)                                    | — standalone         |
| WorkoutPlan               | (root, TraineeId, IsActive)                                     |                      |
| └ WorkoutDay              | (ScheduledDay: DayOfWeek?, Order)                               |                      |
| └ WorkoutExercise         | (→ Exercise, Sets/Reps/WeightKg/RestSeconds/Notes)              |                      |
| NutritionPlan             | (root, TraineeId, IsActive, optional daily Target macros)       |                      |
| └ Meal                    | (Order)                                                         |                      |
| └ MealItem                | (→ Food, Quantity = servings)                                   |                      |
| WorkoutPlanTemplate       | (root; NO TraineeId / NO IsActive)                              | — reusable blueprint |
| └ WorkoutTemplateDay      | (ScheduledDay carried onto clones)                              |                      |
| └ WorkoutTemplateExercise | (→ Exercise)                                                    |                      |
| NutritionPlanTemplate     | (root; NO TraineeId / NO IsActive; targets carried onto clones) |                      |
| └ NutritionTemplateMeal   |                                                                 |                      |
| └ NutritionTemplateItem   | (→ Food)                                                        |                      |
| WorkoutLog                | (root, TraineeId; optional WorkoutPlanId + WorkoutDayId, Date)  |                      |
| └ WorkoutLogEntry         | (→ Exercise; actuals + PRESCRIBED snapshot)                     |                      |
| NutritionLog              | (root, TraineeId; optional NutritionPlanId, Date)               |                      |
| └ NutritionLogEntry       | (→ Food, Quantity)                                              |                      |
| ProgressEntry             | (root, TraineeId; weight/bodyfat/measurements/date)             | — standalone         |
| TraineeNote               | (root, TraineeId; Date, Text; author = CreatorId)               | — standalone         |

**Cross-aggregate links are Guid FKs only** — never navigation across an aggregate boundary.

`WorkoutExercise/WorkoutLogEntry/WorkoutTemplateExercise → Exercise;`

`MealItem/NutritionLogEntry/NutritionTemplateItem → Food;` all trainee-owned roots → `Trainee` via `TraineeId`.

## 4.3 Key entity notes

- `Trainee` — a coaching profile 1:1 with an ABP `IdentityUser` (role `Trainee`) via `UserId`.

Credentials live on the user; coaching data lives here. `UserName` is a snapshot of the login name. `Activate()/Deactivate()` toggle `IsActive`.

- `WorkoutDay.ScheduledDay (DayOfWeek?)` — drives the trainee "Today" view. `null` = never

surfaces as today; a weekday with no scheduled day = rest day.

- `WorkoutLogEntry` — stores **actuals** (`Sets/Reps/WeightKg/Notes`) \*and\* a \*\*prescribed

snapshot\*\* (`PrescribedSets/PrescribedReps/PrescribedWeightKg`, nullable). The snapshot is captured when logging from a plan so planned-vs-done survives later plan edits. Manual logs have null prescribed.

- `MealItem/NutritionLogEntry` store only `FoodId` + `Quantity` (servings). Calories & macros

are **computed from the Food library at read time**, never stored on the item — so editing a food's macros retroactively corrects everything.

- **Templates** mirror plans but drop `TraineeId` and `IsActive` (no active concept for a blueprint),

and reuse the plan's `*Consts` (no dedicated consts classes).

#### 4.4 Enums (`CoachApp.Enums`, stored as `byte`)

- **Gender:** `Unspecified=0`, `Male=1`, `Female=2`
- **TrainingGoal:** `General=0`, `LoseWeight=1`, `BuildMuscle=2`, `Maintain=3`, `ImproveFitness=4`, `Strength=5`
- **MuscleGroup:** `Other=0`, `Chest=1`, `Back=2`, `Shoulders=3`, `Arms=4`, `Legs=5`, `Core=6`, `FullBody=7`, `Cardio=8`
- **Equipment:** `None=0`, `Bodyweight=1`, `Barbell=2`, `Dumbbell=3`, `Machine=4`, `Cable=5`, `Kettlebell=6`, `ResistanceBand=7`, `Other=8`
- Plus `System.DayOfWeek` (used by `ScheduledDay`).

#### 4.5 Consts (field length limits — `Domain.Shared/Entities/.../*Consts.cs`)

`TraineeConsts` (Username 256, First/Last 64, Email 256, Phone 32, Password 6–128),  
`ExerciseConsts` (Name 128, Desc 512, Instructions 2000, MediaUrl 512), `FoodConsts` (Name 128, Desc 512, ServingUnit 16), `WorkoutPlanConsts` (Name 128, Desc 512, DayName 64, Reps 32, Notes 512 — also used by templates), `NutritionPlanConsts` (Name 128, Desc 512, MealName 64 — also used by templates), `WorkoutLogConsts` (Reps 32, Notes 512, **MaxEntries 100**), `NutritionLogConsts` (Notes 512, **MaxEntries 100**), `ProgressEntryConsts` (Notes 512), `TraineeNoteConsts` (Text 2000).

---

## 5. Persistence (EF Core)

---

- One `CoachAppDbContext` (`src/CoachApp.EntityFrameworkCore/EntityFrameworkCore/`). It replaces

the Identity and TenantManagement DbContexts (`[ReplaceDbContext]`) so you can JOIN across them. Connection string name `Default`; table prefix `CoachAppConsts.DbTablePrefix = "App"`, null schema.

- **21 domain DbSets** are declared (roots + children): Trainees, Exercises, Foods, WorkoutPlans /

WorkoutDays / WorkoutExercises, WorkoutPlanTemplates / WorkoutTemplateDays / WorkoutTemplateExercises, WorkoutLogs / WorkoutLogEntries, NutritionPlans / Meals / MealItems, NutritionPlanTemplates / NutritionTemplateMeals / NutritionTemplateItems, NutritionLogs / NutritionLogEntries, ProgressEntries, TraineeNotes.

- Each entity has an `IEntityTypeConfiguration<T>` under `EntityConfigurations/<Feature>/`, all

auto-applied via `builder.ApplyConfigurationsFromAssembly(...)` in `OnModelCreating`.

- Migrations live in `CoachApp.EntityFrameworkCore/Migrations/`.

---

## 6. Mapping (Mapster)

---

- `CoachAppMapsterConfig.Configure()` — one `TypeAdapterConfig<Entity, Dto>.NewConfig()` line per

mapping, **entity** → **DTO only**. Create/Update flow through the aggregate's behavior methods, never through mapping.

- `MapsterObjectMapper : IAutoObjectMappingProvider` — routes ABP's `ObjectMapper.Map<...>()`

through Mapster. Registered in `CoachAppApplicationModule`.

- **Enrichment** is separate from mapping: cross-aggregate display values (exercise names, food names,

macro totals) are filled in the app service via dedicated helpers — `WorkoutLogEnricher`, `NutritionPlanEnricher`, `NutritionLogEnricher`, `NutritionPlanTemplateEnricher` — that batch-load the referenced library rows. Per-serving macro math lives in exactly one place per domain.

---

## 7. Auth, identity & multi-tenancy

---

### 7.1 OpenIddict (`Domain/OpenIddict/OpenIddictDataSeedContributor.cs`)

- One API **scope**: `CoachApp`.
- Two **clients** (seeded only if their ClientId is configured):
- `CoachApp_App` — public client enabling **password grant (ROPC)** plus `authorization_code`,

`client_credentials`, `refresh_token`, and ABP's LinkLogin/Impersonation grants. This is how the SPA/ mobile app logs a coach or trainee in with `username+password`.

- `CoachApp_Swagger` — `authorization_code` only, for the Swagger UI.
- Host (`CoachAppHttpApiHostModule`): OpenIddict **validation** with audience `CoachApp`,

`UseLocalServer()` — same process is auth server \*and\* resource server. **Dynamic claims enabled** (`IsDynamicClaimsEnabled = true`), so permission/role changes take effect without re-login. In production it loads a real `openiddict.pfx` (passphrase from `AuthServer:CertificatePassPhrase`).

### 7.2 Roles & permissions seeding

- Role name constants (`Domain.Shared/CoachAppRoles.cs`): `Coach`, `Trainee`.
- Permission prefixes (`Domain.Shared/CoachAppPermissionPrefixes.cs`): `CoachApp.Coach`,

`CoachApp.Trainee` (duplicated here because Domain can't reference the Contracts permissions class).

- `Domain/Data/CoachAppDataSeedContributor.cs` runs for the host and every tenant, and for each

ensures the `Coach` and `Trainee` roles exist (tenant-scoped) and grants the **\*\*entire `Coach.*` subtree to `Coach` and entire `Trainee.*` subtree\*\*** to Trainee.

- **No custom admin seeding** — the default admin comes from ABP's `IdentityDataSeedContributor`,

driven by `CoachAppDbMigrationService.SeedDataAsync` (admin email/password via `DataSeedContext`). Defaults: `admin@abp.io` / `1q2w3E*` (`CoachAppConsts`).

- A **demo tenant** (`"demo"`) is created only when `CoachApp:SeedDemoTenant == true` (set in the

**DbMigrator** appsettings; must be false/absent in production).

### 7.3 Multi-tenancy — "tenant-per-coach"

- `MultiTenancyConsts.IsEnabled = true`; host calls `app.UseMultiTenancy()`.
- **Each coach = one tenant.** The tenant's auto-seeded admin \*is\* the coach. Trainees are additional

`IdentityUsers` (role `Trainee`) created inside that tenant, each linked 1:1 to a `Trainee` profile via `UserId`.

- **One-coach-per-tenant guard:** `Domain/Identity/SingleCoachPerTenantHandler.cs` — a local event

handler on `IdentityUser` create/update. If a change would make a second user a Coach in the tenant, it throws `BusinessException(SecondCoachNotAllowed)` and rolls back the unit of work. It short-circuits on login/lockout/profile hot paths.

- **Current tenant/user** resolved by ABP from the token in the pipeline; use `ICurrentTenant` /

`ICurrentUser`. Seeders switch context with `_currentTenant.Change(tenantId)`.

### 7.4 Permission tree (`Application.Contracts/Permissions/CoachAppPermissions.cs`)

Single group `CoachApp`. CRUD nodes built by an `AddCrud` helper (`.Create/.Update/.Delete`).

\*\*Coach persona (`CoachApp.Coach.*`)\*\*

| Node                                                                                                                                                                            | Children                                                                         |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| <code>Trainees</code>                                                                                                                                                           | Create, Update, Delete, <b>ResetPassword</b>                                     |
| <code>Exercises</code> , <code>WorkoutPlans</code> , <code>WorkoutPlanTemplates</code> , <code>Foods</code> , <code>NutritionPlans</code> , <code>NutritionPlanTemplates</code> | Create, Update, Delete                                                           |
| <code>Progress</code> , <code>Notes</code>                                                                                                                                      | Create, Update, Delete                                                           |
| <code>Tracking</code>                                                                                                                                                           | *(leaf — read-only; shared by WorkoutLog, NutritionLog, TraineeDashboard reads)* |

**\*\*Trainee persona (`CoachApp.Trainee.*`)\*\***

| Node                                                                                                                                                          | Children                                                        |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| <code>MyProfile</code> , <code>MyDashboard</code> , <code>MyToday</code> , <code>MyWorkoutPlans</code> , <code>MyNutritionPlans</code> , <code>MyNotes</code> | *(leaf)*                                                        |
| <code>WorkoutLogs</code> , <code>NutritionLogs</code>                                                                                                         | Create, Update *(no Delete — delete rides on the leaf/Default)* |
| <code>MyProgress</code>                                                                                                                                       | Create *(Update reuses Create; Delete rides on Default)*        |

The asymmetry is intentional: coaches have full CRUD over shared libraries/plans/trainees; trainees have read + append-only logging (they log activity but cannot delete history).

## 7.5 Error codes (`Domain.Shared/CoachAppDomainErrorCodes.cs`)

Mapped to localization via `MapCodeNamespace("CoachApp", typeof(CoachAppResource))`. Throw as `BusinessException(code)`; clients get a stable code + localized message.

| Code                        | Constant                       | Meaning / where enforced                                          |
|-----------------------------|--------------------------------|-------------------------------------------------------------------|
| <code>CoachApp:00001</code> | <code>TraineeNotFound</code>   | Referencing a missing trainee (plan/entry/note/dashboard)         |
| <code>CoachApp:00002</code> | <code>ExercisesNotFound</code> | A referenced exercise doesn't exist                               |
| <code>CoachApp:00003</code> | <code>FoodsNotFound</code>     | A referenced food doesn't exist                                   |
| <code>CoachApp:00004</code> | <code>ExerciseInUse</code>     | Delete blocked — used by a plan/template/log (deactivate instead) |

| Code                        | Constant                                       | Meaning / where enforced                        |
|-----------------------------|------------------------------------------------|-------------------------------------------------|
| <code>CoachApp:00005</code> | <code>FoodInUse</code>                         | Delete blocked — used by a plan/template/log    |
| <code>CoachApp:00006</code> | <code>DuplicateScheduledDay</code>             | Two workout days on the same weekday            |
| <code>CoachApp:00007</code> | <code>SecondCoachNotAllowed</code>             | One-coach-per-tenant (identity guard)           |
| <code>CoachApp:00008</code> | <code>CannotModifyCoachAuthoredProgress</code> | Trainee may only edit/delete their own progress |

**Localization:** `en.json` and `ar.json` are the maintained pair (both fully cover error codes + permission labels + roles + enums). Other language stubs exist but are mostly untranslated.

## 8. Coach-side services (`Apis/<Feature>/`, no `My` prefix)

All derive from `CoachAppAppService`, are tenant-scoped automatically, gate reads with a class-level `[Authorize(...Default)]` and writes with per-method `[Authorize(...Create/Update/Delete)]`. Queries are inline (`GetQueryableAsync` + `WhereIf` + `Dynamic.Core.OrderBy(sorting)` + paging + `AsyncExecuter`). `TraineeId` is **immutable on update** (a plan/entry/note can never be re-parented).

| Service                   | Permission root              | Highlights                                                                                                                              |
|---------------------------|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <b>TraineeAppService</b>  | <code>Coach.Trainees</code>  | <b>Provisions login + profile</b> (see 8.1). Delete = recoverable deactivation. <code>ResetPasswordAsync</code> . Roster search/filter. |
| <b>ExerciseAppService</b> | <code>Coach.Exercises</code> | Library CRUD. Delete blocked if referenced ( <code>ExerciseInUse</code> ).                                                              |

| Service                                | Permission root                           | Highlights                                                                                                                                                                                          |
|----------------------------------------|-------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>FoodAppService</b>                  | <code>Coach.Foods</code>                  | Library CRUD (per-serving macros). Delete blocked if referenced ( <code>FoodInUse</code> ).                                                                                                         |
| <b>WorkoutPlanAppService</b>           | <code>Coach.WorkoutPlans</code>           | Plan→Days→Exercises. Validates trainee + exercises exist + <b>unique scheduled weekday</b> . <code>SetActiveAsync</code> enforces single active plan per trainee. Full-replace structure on update. |
| <b>NutritionPlanAppService</b>         | <code>Coach.NutritionPlans</code>         | Plan→Meals→Items + optional targets. Validates trainee + foods. Single-active per trainee.                                                                                                          |
| <b>WorkoutPlanTemplateAppService</b>   | <code>Coach.WorkoutPlanTemplates</code>   | Template CRUD. <code>CloneToTraineeAsync</code> → new <b>inactive</b> WorkoutPlan (guarded by <code>WorkoutPlans.Create</code> ). <code>SaveAsTemplateAsync</code> snapshots a plan.                |
| <b>NutritionPlanTemplateAppService</b> | <code>Coach.NutritionPlanTemplates</code> | Analog of the workout template service (clone/save-as-template, carries targets).                                                                                                                   |
| <b>WorkoutLogAppService</b>            | <code>Coach.Tracking</code>               | <b>Read-only.</b> View a trainee's logs by date range (enriched with exercise names).                                                                                                               |

| Service                           | Permission root             | Highlights                                                                |
|-----------------------------------|-----------------------------|---------------------------------------------------------------------------|
| <b>NutritionLogAppService</b>     | <code>Coach.Tracking</code> | <b>Read-only.</b> View a trainee's nutrition logs (enriched with macros). |
| <b>ProgressEntryAppService</b>    | <code>Coach.Progress</code> | Full CRUD over a trainee's body-metrics timeline.                         |
| <b>TraineeNoteAppService</b>      | <code>Coach.Notes</code>    | CRUD over dated per-trainee notes.                                        |
| <b>TraineeDashboardAppService</b> | <code>Coach.Tracking</code> | Read-only analytics; delegates math to <code>DashboardCalculator</code> . |

## 8.1 How a trainee login is created (`TraineeAppService.CreateAsync`)

Injects `IRepository<Trainee, Guid>` + `IdentityUserManager`. In one call:

1. Create an `IdentityUser` in `CurrentTenant.Id` (email defaults to `{UserName}@coachapp.local`).
2. `CreateAsync(user, password)` → `AddToRoleAsync(user, "Trainee")`. Identity failures surface as

`UserFriendlyException` (via `CheckIdentityErrors`).

1. Build the profile with `Trainee.Create(...)` bound to `user.Id`; insert.
2. `ApplyLoginStateAsync(user, IsActive)` — active clears logout; **inactive sets permanent logout**

so the trainee can't sign in. Called from Create/Update/Delete.

`DeleteAsync` is a **recoverable deactivation** (sets `IsActive=false` + locks login; history kept; reactivate via `UpdateAsync`). `ResetPasswordAsync` does `RemovePasswordAsync` + `AddPasswordAsync` in one unit of work (no current password required; username unchanged).

## 8.2 DashboardCalculator (shared, `Apis/Dashboards/DashboardCalculator.cs`, `internal static`)

Used by **both** the coach dashboard and the trainee dashboard (only difference: coach passes a trainee id from input, trainee resolves it from the current user).

- **Nutrition adherence (day):** sum consumed macros across \*all\* logs on the date; target = active

plan's explicit targets, else computed meal totals;  $\text{percent} = \text{consumed} / \text{target} * 100$  (1dp, **uncapped**, null if no plan/zero target).  $\text{OverallPercent} = \text{CaloriesPercent}$ .

- **Nutrition adherence (range):**  $\text{consumedTotal} / (\text{targetPerDay} * \text{daysLogged}) * 100$  — averaged over

days actually logged.

- **Workout completion:**  $\text{CompletedSessions} = \text{distinct days with a log in range}$ ;  $\text{PlannedPerWeek} =$

distinct scheduled weekdays of the active plan;  $\text{Weeks} = \text{ceil}(\text{days} / 7)$ ;  $\text{CompletionPercent} = \text{completed} / \text{planned} * 100$  (uncapped, null if no active plan).

---

## 9. Trainee-side services (*Apis/<Feature>/*, *My* prefix)

---

### 9.1 The "who am I" base — *MyTraineeAppServiceBase*

*src/CoachApp.Application/MyTraineeAppServiceBase.cs* (extends *CoachAppAppService*). **\*\*The single place\*\*** the logged-in user maps to a Trainee:

- *GetCurrentTraineeAsync()* / *GetCurrentTraineeIdAsync()* — look up *Trainee* where

*UserId == CurrentUser.GetId()*. No Trainee row → *EntityNotFoundException(typeof(Trainee))*.

- **Security invariant:** the trainee id is **\*\*always** derived from the current user, never from client

input. **\*\*** No trainee-facing Create/Update DTO carries a *TraineeId*. Cross-trainee access is uniformly a **404** (foreign id → *EntityNotFoundException*) so ownership is never disclosed.

### 9.2 Services

| Service                          | Permission                            | What it does                                                                                                                                                    |
|----------------------------------|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>MyProfileAppService</b>       | <code>Trainee.MyProfile</code>        | Read own profile; <b>restricted</b> self-edit — only <code>PhoneNumber/Email/BirthDate</code> (goal/weights/active/username stay coach-owned).                  |
| <b>MyWorkoutPlanAppService</b>   | <code>Trainee.MyWorkoutPlans</code>   | Read-only. List/get/getActive — <b>active plans only</b> (inactive hidden). Enriches exercise names.                                                            |
| <b>MyNutritionPlanAppService</b> | <code>Trainee.MyNutritionPlans</code> | Read-only mirror; active plans only; enriches meals/macros.                                                                                                     |
| <b>MyWorkoutLogAppService</b>    | <code>Trainee.WorkoutLogs</code>      | Create (manual) / <b>CreateFromDay</b> / list / get / update / delete. Validates plan+day ownership and exercises. Preserves prescribed snapshot on update.     |
| <b>MyNutritionLogAppService</b>  | <code>Trainee.NutritionLogs</code>    | Create (manual) / <b>CreateFromPlan</b> / list / get / update / delete. Validates plan ownership and foods.                                                     |
| <b>MyProgressAppService</b>      | <code>Trainee.MyProgress</code>       | Create/update/delete own entries + list/get. <code>IsCoachAuthored</code> flag; can't modify coach-authored ( <code>CannotModifyCoachAuthoredProgress</code> ). |
| <b>MyNoteAppService</b>          | <code>Trainee.MyNotes</code>          | Read-only — read coach's notes addressed to this trainee.                                                                                                       |
| <b>MyTodayAppService</b>         | <code>Trainee.MyToday</code>          | The composed "Today" home screen (see 9.3).                                                                                                                     |

| Service                      | Permission                       | What it does                                                                                        |
|------------------------------|----------------------------------|-----------------------------------------------------------------------------------------------------|
| <b>MyDashboardAppService</b> | <code>Trainee.MyDashboard</code> | Own nutrition adherence (day + weekly) & workout completion, via <code>DashboardCalculator</code> . |

### 9.3 The "Today" view (`MyTodayAppService.GetAsync`)

Input carries an optional **client-local** `Date` (so "today" respects the trainee's timezone; null → `UtcNow.Date`). Everything resolved for the current trainee. It composes:

- **Workout:** active plan → today's scheduled days (``Days.Where(d => d.ScheduledDay == date.DayOfWeek)``), `IsRestDay`` (active plan but nothing scheduled today), and the most-recent workout log for the date (`AlreadyLoggedWorkoutToday`, `LatestWorkoutLog`).

- **Nutrition:** active plan (meals+targets) + `NutritionAdherence` for the day (via `DashboardCalculator`) + `AlreadyLoggedNutritionToday`.

- Date is clamped to `[2000-01-01, 2100-01-01]` to keep day-range math off `DateTime` overflow.

### 9.4 The "log from plan" flows (server-side snapshots, `Create`-permissioned)

- `CreateFromDayAsync(CreateWorkoutLogFromDayDto)` — DTO is just `{WorkoutDayId, Date, Notes?}`.

Ownership resolved **through the plan aggregate root** (``TraineeId == me && Days.Any(d => d.Id == WorkoutDayId)``; foreign → 404, parent id never leaked). Each prescribed exercise becomes a log entry whose **actuals are seeded from, and prescribed snapshot equals, the plan's Sets/Reps/WeightKg**.

- `CreateFromPlanAsync(CreateNutritionLogFromPlanDto)` — `{NutritionPlanId, Date, Notes?}`;

ownership check, then **flatten** meals→items into ordered entries (`FoodId` + `Quantity`).

- **Prescribed-snapshot preservation on update:** `UpdateAsync` clears and rebuilds entries from

client input, but the prescribed values are captured beforehand keyed by `(ExerciseId, Order)` and restored — client input can never overwrite the prescription record.

## 10. Cross-cutting rules & where they live

| Rule                                          | Enforced in                                                                                           |
|-----------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Tenant isolation (coach sees only own data)   | ABP tenant filter on every <code>IMultiTenant</code> root                                             |
| One coach per tenant                          | <code>SingleCoachPerTenantHandler</code> (identity event) → <code>SecondCoachNotAllowed</code>        |
| One active workout/nutrition plan per trainee | Plan app services ( <code>SetActiveInternalAsync</code> )                                             |
| At most one scheduled workout per weekday     | <code>WorkoutPlanAppService.ValidateScheduledDaysUnique</code> → <code>DuplicateScheduledDay</code>   |
| Referenced trainee/exercise/food must exist   | App-service existence checks → <code>TraineeNotFound/ExercisesNotFound/FoodsNotFound</code>           |
| Can't delete a library item that's in use     | Exercise/Food delete guards → <code>ExerciseInUse/FoodInUse</code>                                    |
| Trainee only touches own data                 | <code>MyTraineeAppServiceBase</code> + <code>TraineeId==me</code> filters (foreign → 404)             |
| Trainee can't edit coach-authored progress    | <code>MyProgressAppService.EnsureSelfAuthored</code> → <code>CannotModifyCoachAuthoredProgress</code> |

| Rule                                       | Enforced in                                                                       |
|--------------------------------------------|-----------------------------------------------------------------------------------|
| <code>TraineeId</code> immutable on update | Coach-side update methods ignore incoming <code>TraineeId</code>                  |
| Prescribed-vs-actual snapshot durability   | <code>WorkoutLog</code> entries + <code>MyWorkoutLog</code> update restore-by-key |
| Field lengths / required fields            | Entity <code>Check.*</code> against <code>*Consts</code>                          |

## 11. Build, run, migrate, test

```
# Build the whole solution (must be clean)
dotnet build CoachApp.slnx

# Run the API + Swagger
dotnet run --project src/CoachApp.HttpApi.Host          # https://localhost:44370/swagger

# Tests (xUnit + Shouldly)
dotnet test

# Add a migration (writing files is always fine)
dotnet ef migrations add <Name> \
  --project src/CoachApp.EntityFrameworkCore \
  --startup-project src/CoachApp.EntityFrameworkCore

# Apply migrations + seed roles/permissions/admin (⚠ touches the DB)
dotnet run --project src/CoachApp.DbMigrator
```

⚠ **\*\*Applying a migration or running DbMigrator touches the database — always confirm with the team before doing so.\*\*** Generating/writing migration files is fine; executing them is not automatic.

Default dev connection string (`HttpApi.Host/appsettings.json`): LocalDB, database `CoachApp`.  
Auth authority `https://localhost:44370`, Swagger client `CoachApp_Swagger`.

## 12. Adding a feature (vertical slice, bottom-up)

Mirror the **Trainee / WorkoutPlan** slice:

1. `Domain.Shared` → `Entites/<Feature>/<Entity>Consts.cs`, any enum in `Enums/`, error code in

`CoachAppDomainErrorCodes`.

1. `Domain` → `Entites/<Feature>/<Entity>.cs` (root: `FullAuditedAggregateRoot<Guid>` + `IMultiTenant`,

`static Create`, behavior methods; children via `Add*`).

1. `Application.Contracts` → `Entites/<Feature>/` DTOs (`FullAuditedEntityDto<Guid>` out, `Create/Update`

in, `Get...ListInput : PagedAndSortedResultRequestDto`) + `I<Entity>AppService`.

1. `Application.Contracts/Permissions` → add nodes to `CoachAppPermissions` +

`CoachAppPermissionDefinitionProvider` (Coach and/or Trainee subtree; the seeder auto-grants them).

1. `Application` → `Apis/<Feature>/<Entity>AppService.cs` (fat, inline queries, `[Authorize]`,

existence/uniqueness checks) + one `CoachAppMapsterConfig` line (+ enricher if it needs cross-aggregate display data). Trainee-facing? extend `MyTraineeAppServiceBase`.

1. `EntityFrameworkCore` → `EntityConfigurations/<Feature>/<Entity>Configuration.cs`, add `DbSets`,

`dotnet ef migrations add`.

1. Localization → keys in **both** `en.json` and `ar.json`.
2. Tests → mirror `test/CoachApp.*.Tests`.

The detailed checklist and the specialist subagents live in `.claude/agents/` (`coachapp-architect`, `-backend`, `-db`, `-tester`, `-reviewer`, `-security`, `-runner`) and the rules in `.cursor/rules/**/*mdc`.

---

## 13. Notable design decisions & open items

---

- **Delete = deactivate** for trainees and library items (recoverable; history preserved). A hard

purge is deferred (noted V1.1).

- **Trainees see active plans only** — inactive/archived/draft plans are hidden from the trainee side

(deferred to V1.1).

- **Percentages are uncapped** on dashboards — going over target stays visible (not clamped to 100%).
- **Multiple logs per day are allowed**; the Today view surfaces the most recent, dashboards sum them.
- **Nutrition macros are always computed from the Food library at read time** — never denormalized

onto plan/log items — so a food correction propagates everywhere.

- **Prescribed vs actual** is a first-class concept only on **workout** logs; nutrition "from plan"

simply becomes the actual entries (no prescribed concept).

- **en + ar** are the fully maintained localizations; the other language JSON stubs are largely untranslated defaults.

- **The Entites spelling** (typo) is load-bearing across namespaces and folders — keep it.